

REPORTE TAREA 2 y 3

ALGORITMOS Y COMPLEJIDAD

«Explorando la Distancia entre Cadenas, una Operación a la Vez»

Lucas Mosquera

18 de noviembre de 2024

01:28

Resumen

En este informe se busca comparar la eficiencia temporal de los algoritmos de fuerza bruta y programación dinámica en un problema de distancia mínima de edición modificado que agrega transposiciones y costos variables.

Esto se logra a través de evaluar distintos datasets de strings de largos del 1 al 15 con estos algoritmos, comparando sus tiempos de ejecución.

*Se llega a la conclusión de que aún cuando fuerza bruta es más eficiente para strings de largos menores a 5, programación dinámica es un algoritmo mucho más eficiente en complejidad temporal, pasando de $O(4^{\min(n,m)})$ a $O(n * m)$, valiendo la pena al memoria extra $O(n * m)$ que conlleva el uso de programación dinámica para este problema.*

Índice

1. Introducción	2
2. Diseño y Análisis de Algoritmos	3
3. Implementaciones	8
4. Experimentos	9
5. Conclusiones	13
6. Condiciones de entrega	14
A. Apéndice 1	15

1. Introducción

Este informe está centrado en el área de Análisis y Diseño de algoritmos en Ciencias de la Computación. Se buscará responder a la pregunta de cual es el algoritmo más eficiente en el ámbito temporal y espacial para resolver el problema de distancia mínima de edición modificado, comparando entre el algoritmo de Fuerza Bruta y el algoritmo de Programación Dinámica.

El problema con el que se experimentará en este informe es una versión modificada de la distancia mínima de edición, la versión original trata sobre computar la cantidad mínima de acciones para convertir un string original en un string destino, estas acciones son insertar, eliminar y reemplazar, teniendo costos 1, 1 y 2 respectivamente. La versión modificada añade una acción más, la transposición en la que se puede intercambiar un carácter con el siguiente, además de agregar costos variables dependiendo de con que carácter se esté tratando.

Este problema es relevante resolverlo de la manera más eficiente posible ya que estos algoritmos pueden ser ocupados por ejemplo en el área de genética, donde se busca transformar una cadena de bases a otra.

Se ocuparon dos algoritmos distintos para la resolución del problema, el primero es el de Fuerza Bruta, que es ver todas los caminos posibles de acciones y quedarse con la secuencia de acciones que minimice el costo, por otro lado está Programación Dinámica, que básicamente es Fuerza Bruta inteligente, donde se guardan el resultado de los subproblemas al computarlos, para no tener que volver a computarlos de nuevo al volver a ese subproblema.

El objetivo de este informe va a ser determinar cual de los algoritmos propuestos es más eficiente en complejidad temporal y en complejidad espacial, además de ver como estos algoritmos actúan en distintos casos del problema general.

2. Diseño y Análisis de Algoritmos

Los pseudocódigos los he diseñado utilizando el paquete *Algorithm2e documentation* [2] para la presentación de algoritmos. Se recomienda consultar *Algorithm2e on CTAN* [3] y *Writing Algorithms in LaTeX* [5].

2.1. Fuerza Bruta

“Indeed, brute force is a perfectly good technique in many cases; the real question is, can we use brute force in such a way that we avoid the worst-case behavior?”

— Knuth, 1998 [4]

El algoritmo de fuerza bruta se basa en computar todas las posibles opciones y quedarse con la opción que entregue el mejor resultado, es bastante simple e intuitivo de implementar normalmente, pero no es un algoritmo muy eficiente debido a que normalmente pasar por todos los casos es del orden de $n!$ o 2^n , lo cual no es computablemente muy amigable.

Para el problema de distancia mínima de edición modificado que vemos en este informe la implementación del algoritmo de fuerza bruta se basa en recursión, habiendo una llamada recursiva por cada acción que se puede tomar, insertar, eliminar, sustituir y transponer.

El algoritmo empieza desde el carácter final de S1 y S2, ve si estos caracteres son iguales, si es el caso pasa al carácter anterior de ambos strings, en caso contrario, hace las llamadas recursivas correspondientes para encontrar el costo mínimo de lograr que sean iguales y se queda con la mejor. Hace esto para cada uno de los caracteres por cada una de las llamadas recursivas logrando hacer todos los caminos posibles y quedándose con el costo menor que se encuentre.

Algoritmo 1: Algoritmo de distancia mínima de edición con costos de inserción, eliminación, sustitución y transposición.

```

1 Procedure DISTANCIADeEDICION( $S1, S2, n, m$ )
2   if  $n = 0$  then
3     suma  $\leftarrow 0$ 
4     for  $i \leftarrow 0$  to  $m - 1$  do
5       suma  $\leftarrow$  suma + costo_ins( $S2[i]$ )
6     return suma
7   else if  $m = 0$  then
8     suma  $\leftarrow 0$ 
9     for  $i \leftarrow 0$  to  $n - 1$  do
10      suma  $\leftarrow$  suma + costo_del( $S1[i]$ )
11    return suma
12  else if  $S1[n - 1] = S2[m - 1]$  then
13    return  $S1, S2, n - 1, m - 1$ 
14  else
15    costoInsertar  $\leftarrow$  costo_ins( $S2[m - 1]$ ) + DISTANCIADeEDICION( $S1, S2, n, m - 1$ )
16    costoEliminar  $\leftarrow$  costo_del( $S1[n - 1]$ ) + DISTANCIADeEDICION( $S1, S2, n - 1, m$ )
17    costoReemplazar  $\leftarrow$  costo_sub( $S1[n - 1], S2[m - 1]$ ) + DISTANCIADeEDICION( $S1, S2, n - 1, m - 1$ )
18    costoTransponer  $\leftarrow \infty$ 
19    if  $n \geq 2$  and  $m \geq 2$  and  $S1[n - 1] = S2[m - 2]$  and  $S1[n - 2] = S2[m - 1]$  then
20      costoTransponer  $\leftarrow$  costo_trans( $S1[n - 2], S1[n - 1]$ ) + DISTANCIADeEDICION( $S1, S2, n - 2, m - 2$ )
21    return mín(costoInsertar, costoEliminar, costoReemplazar, costoTransponer)

```

Como ejemplo de ejecución podemos poner a $S1$ como "once" y $S2$ como "nose":

1. Comparación de los caracteres finales:

- $S1[n - 1] = "e", S2[m - 1] = "e" \rightarrow$ Coinciden.
- Llamada recursiva a "onc", "nos", 3, 3.

2. Comparación de "c" (en $S1$) y "s" (en $S2$):

- No coinciden, se calculan los costos de las operaciones:
 - **Inserción:** costo_ins("s") + DISTANCIADeEDICION("onc", "no", 3, 2).
 - **Eliminación:** costo_del("c") + DISTANCIADeEDICION("on", "nos", 2, 3).
 - **Sustitución:** costo_sub("c", "s") + DISTANCIADeEDICION("on", "no", 2, 2).
 - **Transposición:** No se aplica porque no cumple la condición en el algoritmo.

3. Cálculo de las subllamadas recursivas:

- Se continúa resolviendo las llamadas hasta alcanzar las condiciones base ($n = 0$ o $m = 0$), acumulando costos y comparando los resultados parciales para elegir el mínimo.

4. Evaluación de transposición:

- Si en algún paso $S1[n-1]$ y $S1[n-2]$ coinciden con $S2[m-2]$ y $S2[m-1]$ en orden inverso, se evalúa `costo_trans` y se hace la llamada recursiva correspondiente.

Resultado Final: Se seleccionan los costos mínimos calculados en cada paso para determinar el costo total de la distancia de edición.

Sea n el largo de $S1$ y m el largo de $S2$, la complejidad espacial de algoritmo en el input es del orden $O(n+m)$ y no se ocupa memoria extra a parte del stack de las llamadas recursivas. Suponiendo el peor de los casos, donde no hay caracteres entre $S1$ y $S2$ que sean iguales, además de siempre considerar la llamada recursiva generada por la transposición nos da una complejidad temporal del orden de $O(4^{\min(n,m)})$, esto debido a que existen 4 llamadas recursivas, una para cada acción, y se van a llamar tantas veces como el largo del menor string, luego solo existe un loop lineal para llegar al largo del mayor string.

De las modificaciones hechas al problema base, el que más afecta es la adición de la acción de transponer, que al añadir una llamada recursiva extra pasa la complejidad temporal de $O(3^{\min(n,m)})$ a $O(4^{\min(n,m)})$, los costos variables solo afectan a la hora de que uno de los strings es más corto que el otro, ya que, a la hora de rellenar o eliminar los caracteres sobrantes, en vez de ser un tiempo constante, pasa a ser tiempo lineal al tener que ver el costo de cada uno de estos caracteres por separado.

2.2. Programación Dinámica

Dynamic programming is not about filling in tables. It's about smart recursion!

Erickson, 2019 [1]

- 1) Describa la solución recursiva.
- 2) Escriba la relación de recurrencia, incluyendo condiciones y casos base.
- 3) Identifique subproblemas.
- 4) Defina estructura de datos a utilizar y especifique el orden de calculo que realiza su programa que utiliza programación dinámica.

2.2.1. Descripción de la solución recursiva

Si aplicamos Programación Dinamica a este problema, la solución recursiva nos queda practicamente igual que la solución con fuerza bruta, pero beneficiandonos del uso de un diccionario para guardar los costos de los subproblemas para no tener que volver a computarlos y cortar gran parte de las llamadas recursivas que se hacian en fuerza bruta.

Las únicas diferencias con la solución de fuerza bruta es que en caso de que el subproblema actual este computado y guardado en el diccionario, nos ahorramos la recomputación devolviendo el costo guardado, y en caso de que no este computado, computarlo y guardarlo en el diccionario para posterior uso.

Tanto la complejidad temporal como espacial varían a la hora de aplicar programación dinámica, la complejidad temporal pasa a ser del orden de $O(n * m)$, lo cual se puede comprobar con el enfoque bottom up al incluir 2 for para rellenar todos los subproblemas. Por otro lado, la complejidad espacial auxiliar pasa a ser $O(n * m)$ debido al uso de diccionario para guardar los subproblemas ya computados, la complejidad espacial del input se mantiene igual.

2.2.2. Relación de recurrencia

Como casos base primero tenemos cuando tanto $S1$ como $S2$ son strings vacíos, que al ser iguales su distancia mínima de edición es 0. Luego tenemos el caso base de $S1$ siendo el string vacío y $S2$ un string cualquiera de largo m , la distancia mínima de edición en este caso sería el costo de inserción de todos los caracteres de $S2$. Por último y similar al caso anterior se tiene el caso de que $S1$ es un string cualquiera de largo n y $S2$ es el string vacío, luego la distancia mínima de edición es el costo de eliminar todos los caracteres que se encuentran en $S1$.

2.2.3. Identificación de subproblemas

Dado un problema con $S1$ de largo n y $S2$ de largo m como entradas, se tienen los siguientes subproblemas:

- **Inserción:** El costo de inserción del carácter de $S2$ en la posición $m - 1$ más el costo del subproblema $S1[: n], S2[: m - 1]$
- **Eliminación:** El costo de eliminación del carácter de $S1$ en la posición $n - 1$ más el costo del subproblema $S1[: n - 1], S2[: m]$
- **Sustitución:** El costo de sustituir el carácter de $S1$ en la posición $n - 1$ por el carácter de $S2$ en la posición $m - 1$ más el costo del subproblema $S1[: n - 1], S2[: m - 1]$
- **Sustitución:** El costo de transponer el carácter de $S1$ en la posición $n - 1$ por el carácter de $S1$ en la posición $n - 2$ más el costo del subproblema $S1[: n - 2], S2[: m - 2]$

De estos se el mínimo da el resultado de la distancia mínima de edición para el input dado.

2.2.4. Estructura de datos y orden de cálculo

Para la implementación de programación dinámica con enfoque top-down se ocupó un diccionario que relacionaba pares de largos de los strings (n, m) con su costo mínimo de edición.

El orden de cálculo en el enfoque top-down se ve relacionado a la forma en que se traversa el árbol de posibilidades en la implementación, en este caso la primera rama a la que se accede es la rama de inserción, luego la de eliminación, la de sustitución y por último la transposición, por lo que la computación se hará en ese orden, recorriendo el árbol de decisiones con un DFS y cortando las ramas que ya han sido computadas.

2.2.5. Algoritmo utilizando programación dinámica

Algoritmo 2: Algoritmo de distancia mínima de edición con memoización (programación dinámica top-down).

```

1 Procedure DISTANCIADeEDICION( $S1, S2, n, m$ )
2   if  $(n, m)$  está en cache then
3     return cache[ $(n, m)$ ]
4   if  $n = 0$  then
5     suma  $\leftarrow 0$ 
6     for  $i \leftarrow 0$  to  $m - 1$  do
7       suma  $\leftarrow$  suma + costo_ins( $S2[i]$ )
8     cache[ $(n, m)$ ]  $\leftarrow$  suma
9     return suma
10  else if  $m = 0$  then
11    suma  $\leftarrow 0$ 
12    for  $i \leftarrow 0$  to  $n - 1$  do
13      suma  $\leftarrow$  suma + costo_del( $S1[i]$ )
14    cache[ $(n, m)$ ]  $\leftarrow$  suma
15    return suma
16  else if  $S1[n - 1] = S2[m - 1]$  then
17    resultado  $\leftarrow$  DISTANCIADeEDICION( $S1, S2, n - 1, m - 1$ )
18    cache[ $(n, m)$ ]  $\leftarrow$  resultado
19    return resultado
20  else
21    costoInsertar  $\leftarrow$  costo_ins( $S2[m - 1]$ ) + DISTANCIADeEDICION( $S1, S2, n, m - 1$ )
22    costoEliminar  $\leftarrow$  costo_del( $S1[n - 1]$ ) + DISTANCIADeEDICION( $S1, S2, n - 1, m$ )
23    costoReemplazar  $\leftarrow$  costo_sub( $S1[n - 1], S2[m - 1]$ ) + DISTANCIADeEDICION( $S1, S2, n - 1, m - 1$ )
24    costoTransponer  $\leftarrow \infty$ 
25    if  $n \geq 2$  and  $m \geq 2$  and  $S1[n - 1] = S2[m - 2]$  and  $S1[n - 2] = S2[m - 1]$  then
26      costoTransponer  $\leftarrow$  costo_trans( $S1[n - 2], S1[n - 1]$ ) + DISTANCIADeEDICION( $S1, S2, n - 2, m - 2$ )
27    minCost  $\leftarrow$  mín(costoInsertar, costoEliminar, costoReemplazar, costoTransponer)
28    cache[ $(n, m)$ ]  $\leftarrow$  minCost
29    return minCost

```

3. Implementaciones

En la implementación se tiene los siguientes archivos:

- *costos.cpp* y *costos.hpp*: Archivos donde se guardan las funciones de costos que son ocupadas por las implementaciones de los algoritmos, a su vez incluyen una función para cargar las cargas de costos a partir de los archivos de tablas de costos.
- *cost_insert.txt*, *cost_delete.txt*, *cost_replace.txt* y *cost_transpose.txt*: Archivos que contienen los costos de sus respectivas acciones, para inserción y eliminación se ocupó un costo constante de 50, para transposición se ocupó la distancia de las letras a transponer, y para sustitución se ocupó el doble de la distancia de las letras a sustituir. Todos estos archivos fueron generados mediante el script de python que se encuentra en *costos.py*.
- *normal.txt*, *empty.txt*, *length.txt*, *substring.txt* y *transpose.txt*: Archivos de dataset que posteriormente van a ser explicados con mayor detalle, todos ellos generados mediante el script de python *dataset.py*.
- *fuerza_bruta.cpp*: Implementación del algoritmo de fuerza bruta en C++ para solucionar el problema de distancia mínima de edición, toma como input el nombre del archivo de dataset para hacer las pruebas.
- *programacion_dinamica.cpp*: Implementación del algoritmo de programación dinámica enfoque top-down en C++ para solucionar el problema de distancia mínima de edición, toma como input el nombre del archivo de dataset para hacer las pruebas.
- *results.csv*: Archivo CSV que contiene los datos usados en este informe para generar los gráficos.
- *graphs.ipynb*: Jupyter Notebook con el que se generaron los gráficos que se incluyen en este informe.
- *makefile*: Archivo makefile para compilar y correr las implementaciones de los algoritmos, **make build** para compilar ambas implementaciones de código, **make dinamic** para correr el algoritmo de programación dinámica, y **make brute** para correr el algoritmo de fuerza bruta.

4. Experimentos

Todos los experimentos fueron realizados en un portatil con las siguientes características:

- **Procesador:** Intel Core i3-8130U 2.20GHz
- **RAM:** 8 GB RAM DDR4
- **Almacenamiento:** Intel Optane + 932 GB HDD
- **SO:** Windows 11 + WSL 2 Ubuntu 24.04
- **Software:** gcc v13.2.0, make v4.3, python v3.12.3

Cada uno de los test se ejecuto 4 veces, se ignoró la primera ejecución debido a la carga del cache, y se promedió las 3 restantes para obtener los tiempos finales.

Todos los gráficos fueron realizados ocupando jupyter notebooks con matplotlib, pandas y numpy.

4.1. Dataset (casos de prueba)

Se ocuparon los mismos costos de operaciones para todos los datasets, siendo estos los siguientes:

- *cost_insert.txt*: Tiene los costos de inserción de cada carácter, el costo es constante con un valor de 25, se eligió este valor debido a que es la mitad del mayor costo de sustitución, para no promover eliminación y luego inserción en vez de sustitución.
- *cost_delete.txt*: Tiene los costos de eliminación de cada carácter, son lo mismo que inserción, osea un costo constante de 25 para cualquier carácter.
- *cost_transpose.txt*: Tiene los costos de transponer 2 caracteres contiguos en una matriz, el costo es la distancia entre los 2 caracteres en el abecedario.
- *cost_replace.txt*: Tiene los costos de reemplazar un carácter por otro en una matriz, el costo es el doble de la distancia entre los 2 caracteres en el abecedario, es el doble para que se prefiera transponer en vez de reemplazar.

Todos los datasets tienen casos desde largo 1 a largo 15, se eligió como máximo 15 debido a que largos más grandes hace que fuerza bruta sea prácticamente incomputable. Se tienen los siguientes datasets:

- *normal.txt*: Este dataset es el base, son strings S1 y S2 del mismo largo con letras escogidas al azar, normalmente la distancia mínima se alcanza usando reemplazos al ser todo al azar y tener el mismo largo.
- *empty.txt*: Este dataset contiene S1 como strings vacíos y S2 como un string con letras elegidas al azar, aquí la distancia mínima se alcanza insertando todos los caracteres de S2 a S1.

- *length.txt*: Este dataset contiene S1 de la mitad del largo que S2, ambas con letras escogidas al azar, la distancia mínima se consigue con una combinación de insertar y sustituir letras en S1.
- *substring.txt*: Este dataset contiene S1 y S2 como strings con letras al azar del mismo largo, pero ambos tienen un substring igual, por lo que la distancia mínima se ve reducida comparada con el dataset *normal.txt*.
- *transpose.txt*: Este dataset contiene S1 y S2 como string con letras al azar del mismo largo, pero S1 es el resultado de aplicar varias transposiciones a S2, forzando el uso de la acción de transponer, haciendo que la distancia mínima sea un gasto principalmente en eso.

4.2. Resultados

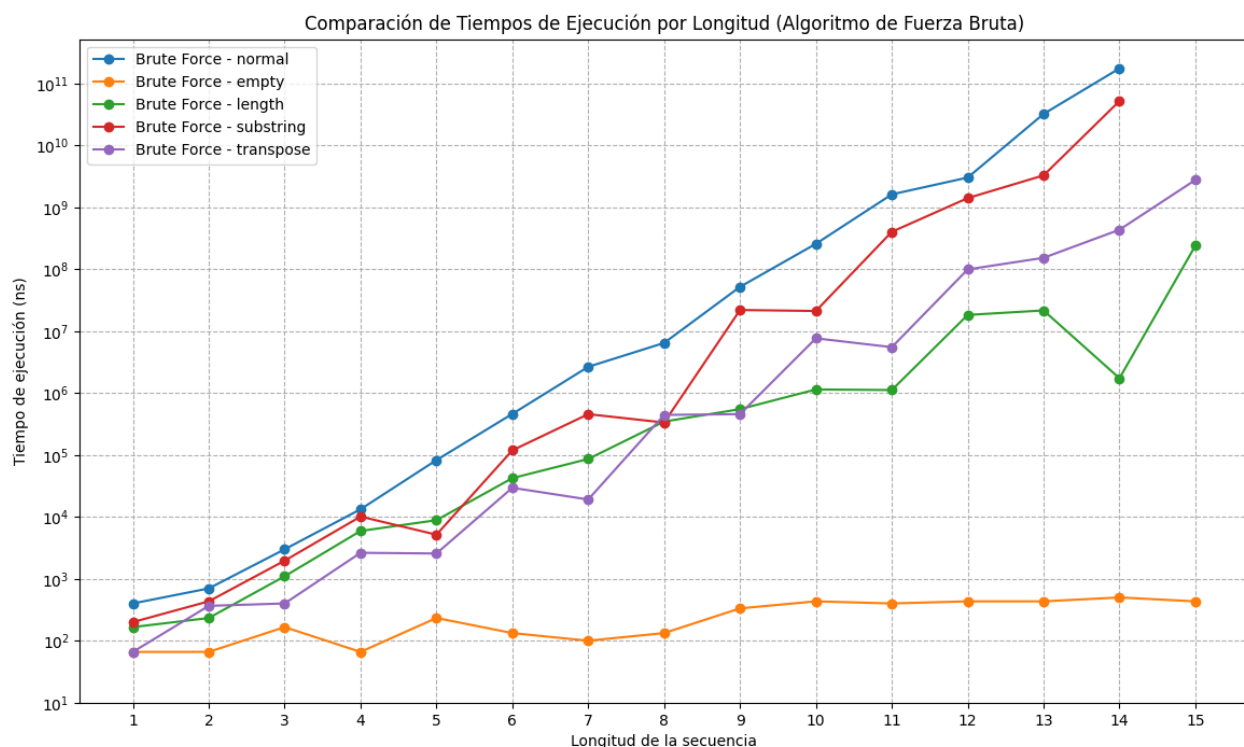


Figura 1: Comparación de tiempos de ejecución entre los distintos datasets usando el algoritmo de fuerza bruta con el aumento del largo, el eje y se encuentra en escala logarítmica para apreciar mejor las diferencias.

En la Figura 1 se puede observar como de forma general los tiempos van aumentando exponencialmente con el aumento de la longitud de los strings, el único dataset que no sigue esta tendencia es el dataset empty, esto debido a que como uno de los strings está vacío solo se debe insertar los caracteres del otro, lo que toma un tiempo lineal. También se puede observar que de los datasets el que se demora más es el dataset normal, los otros datasets con las condiciones impuestas que tienen hacen que el algoritmo tarde un poco menos en ejecutarse.

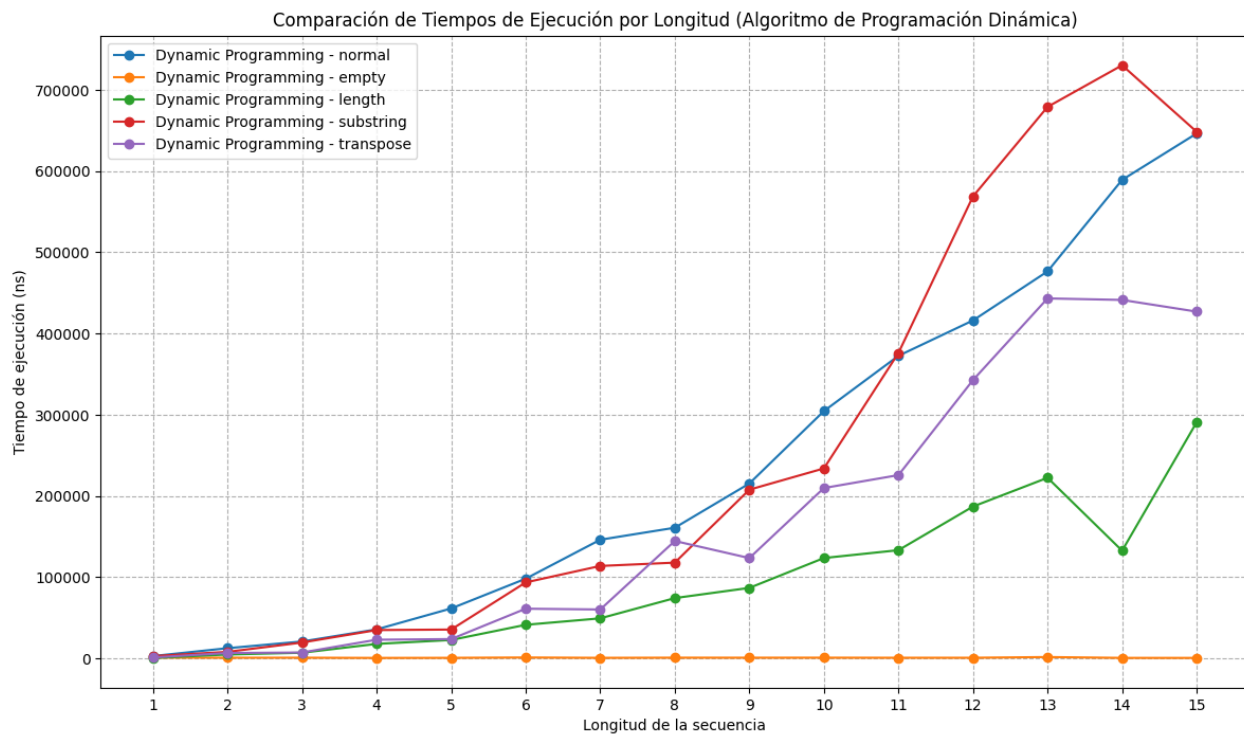


Figura 2: Comparación de tiempos de ejecución entre los distintos datasets usando el algoritmo de programación dinámica con el aumento del largo.

En la Figura 2 se puede observar como de forma general los tiempos van aumentando de forma cuadrática con el aumento de la longitud de los strings, el único dataset que no sigue esta tendencia es el dataset empty, esto debido a que como uno de los strings está vacío solo se debe insertar los caracteres del otro, lo que toma un tiempo lineal. También se puede ver como después del largo 11 el dataset substring empieza a demorar más que el dataset normal, esto se puede deber a que como contiene substrings no se computan tantos subproblemas como en el dataset normal, aunque también se puede deber a variaciones producto del azar en los caracteres de las palabras.

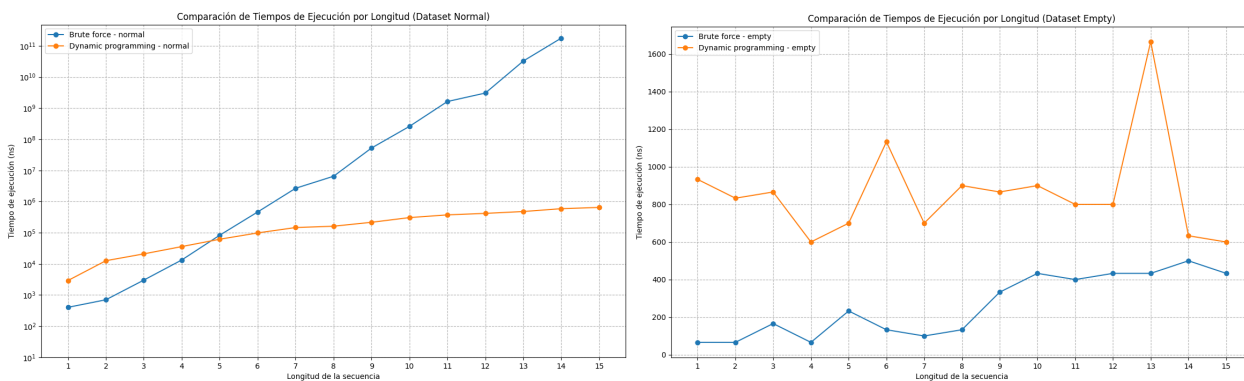


Figura 3: Comparación de tiempos de ejecución entre los distintos algoritmos usando el dataset normal (izquierda) y empty (derecha) con el aumento del largo, el gráfico de la izquierda presenta el eje y en escala logarítmica para apreciar mejor las diferencias.

En la Figura 3 podemos observar como para palabras pequeñas por debajo de 5 caracteres el algoritmo de fuerza bruta es más eficiente en temas temporales que el algoritmo de programación dinámica, esto se puede deber a que no necesita chequear si el subproblema ya ha sido computado, pero ya para largos mayores programación dinámica se mantiene estable, mientras que fuerza bruta crece exponencialmente. También podemos ver en el gráfico de la derecha que para el caso en el que uno de los strings esté vacío, fuerza bruta pasa a ser mejor que programación dinámica, esto porque para este caso base, ambos algoritmos pasan a tener complejidad lineal, solo que programación dinámica de todas formas tiene que chequear la existencia de los subproblemas en su diccionario.

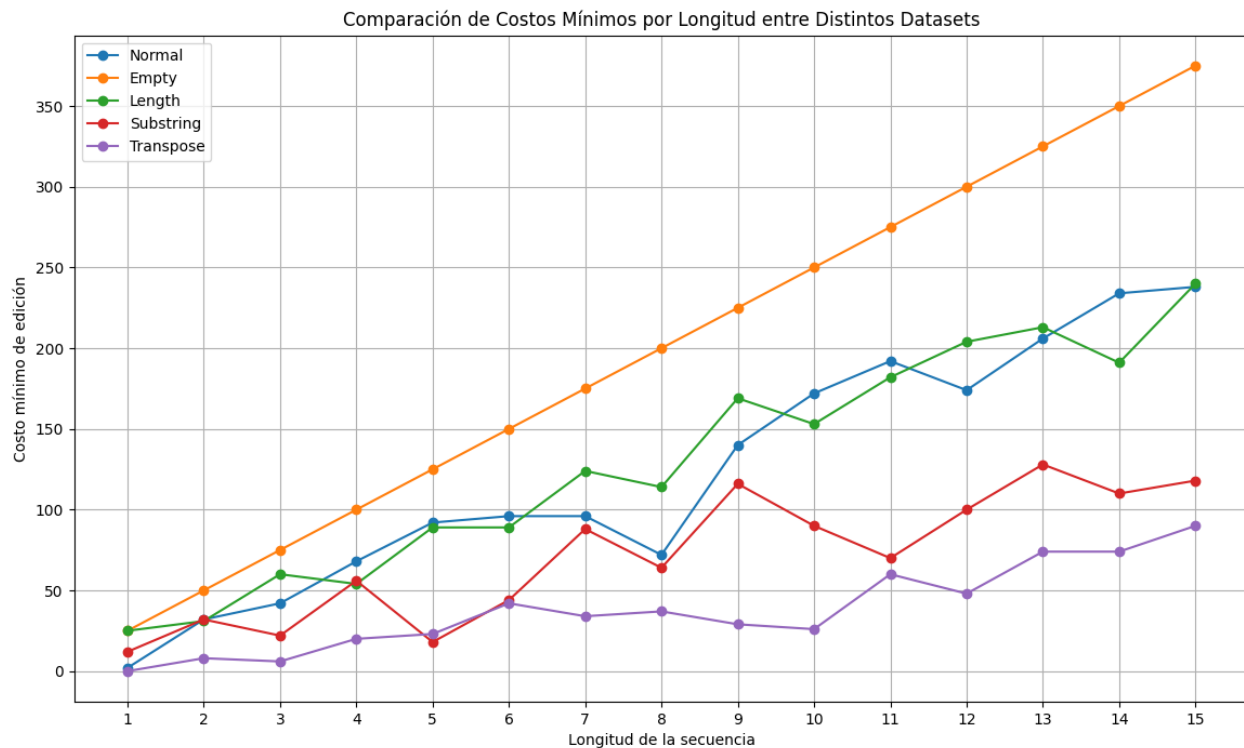


Figura 4: Comparación de costos mínimos entre los distintos datasets con el aumento del largo.

Por último en la Figura 4 podemos ver que con los costos mencionados previamente, la transposición es la que conlleva menos costos de forma general, mientras que la inserción en el dataset empty es bastante costosa, el dataset substring tiene también un costo menor debido a que posee varios caracteres que ya hacen match entre ambos strings, por otro lado el dataset normal tiene principalmente sustituciones y el dataset length tiene sustituciones e inserciones, lo que causa que tengan mayores costos.

5. Conclusiones

La principal conclusión que podemos sacar de los resultados obtenidos y respondiendo a la pregunta inicial del informe, es que aunque programación dinámica puede ser menos eficiente para inputs pequeños debido al uso de memoria extra, para inputs grandes no se compara con fuerza bruta que crece de forma exponencial, la mejora de complejidad temporal de $O(4^{\min(n,m)})$ a $O(n * m)$ es bastante significativa, permitiendo computar tamaños de inputs que con fuerza bruta sería imposible, es verdad que conlleva una complejidad de memoria extra de $O(n * m)$ más el stack de recursión, pero vale la pena la memoria extra para reducir el tiempo de ejecución.

Aparte también podemos concluir que las condiciones del dataset afectan al tiempo de ejecución y a los costos mínimos de edición al favorecer ciertas acciones que o cuestan menos o logran acotar la búsqueda de la solución optima.

6. Condiciones de entrega

- La tarea se realizará **individualmente** (esto es grupos de una persona), sin excepciones.
- La entrega debe realizarse vía <http://aula.usm.cl> en un **tarball** en el área designada al efecto, en el formato **tarea-2 y 3-rol.tar.gz** (rol con dígito verificador y sin guión).

Dicho **tarball** debe contener las fuentes en $\text{\LaTeX}$ (al menos **tarea-2 y 3.tex**) de la parte escrita de su entrega, además de un archivo **tarea-2 y 3.pdf**, correspondiente a la compilación de esas fuentes.
- Si se utiliza algún código, idea, o contenido extraído de otra fuente, este **debe** ser citado en el lugar exacto donde se utilice, en lugar de mencionarlo al final del informe.
- Asegúrese que todas sus entregas tengan sus datos completos: número de la tarea, ramo, semestre, nombre y rol. Puede incluirlas como comentarios en sus fuentes $\text{\LaTeX}$ (en $\text{\TeX}$ comentarios son desde % hasta el final de la línea) o en posibles programas. Anótese como autor de los textos.
- Si usa material adicional al discutido en clases, detállelo. Agregue información suficiente para ubicar ese material (en caso de no tratarse de discusiones con compañeros de curso u otras personas).
- No modifique `preamble.tex`, `tarea_main.tex`, `condiciones.tex`, estructura de directorios, nombres de archivos, configuración del documento, etc. Sólo agregue texto, imágenes, tablas, código, etc. En el código fuente de su informe, no agregue paquetes, ni archivos `.tex` (a excepción de que agregue archivos en `/tikz`, donde puede agregar archivos `.tex` con las fuentes de gráficos en TikZ).
- La fecha límite de entrega es el día **10 de noviembre de 2024**.

NO SE ACEPTARÁN TAREAS FUERA DE PLAZO.

- Nos reservamos el derecho de llamar a interrogación sobre algunas de las tareas entregadas. En tal caso, la nota de la tarea será la obtenida en la interrogación.

NO PRESENTARSE A UN LLAMADO A INTERROGACIÓN SIN JUSTIFICACIÓN PREVIA SIGNIFICA AUTOMÁTICAMENTE NOTA 0.

A. Apéndice 1

Referencias

- [1] Jeff Erickson. *Algorithms*. Jun. de 2019. ISBN: 978-1-792-64483-2.
- [2] Christophe Fiorio. *Algorithm2e documentation*. <http://ctan.math.illinois.edu/macros/latex/contrib/algorithm2e/doc/algorithm2e.pdf>. 2023.
- [3] Christophe Fiorio. *Algorithm2e on CTAN*. <https://ctan.org/pkg/algorithm2e>. 2023.
- [4] Donald E. Knuth. *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN: 0201896850.
- [5] Overleaf. *Writing Algorithms in LaTeX*. <https://www.overleaf.com/learn/latex/Algorithms>. 2023.